

Microservices demo

Github repo: <https://github.com/pesekt1/RabbitMQ-Microservices>

(there are branches for different stages of the architecture)

Contents

Introduction	1
Testing the application.....	2
Testing the asynchronous communication via message broker	2
Testing the synchronous communication via http calls	3
Replacing HTTP call for Like with message queue	3
Scaling and load balancing.....	4

Introduction

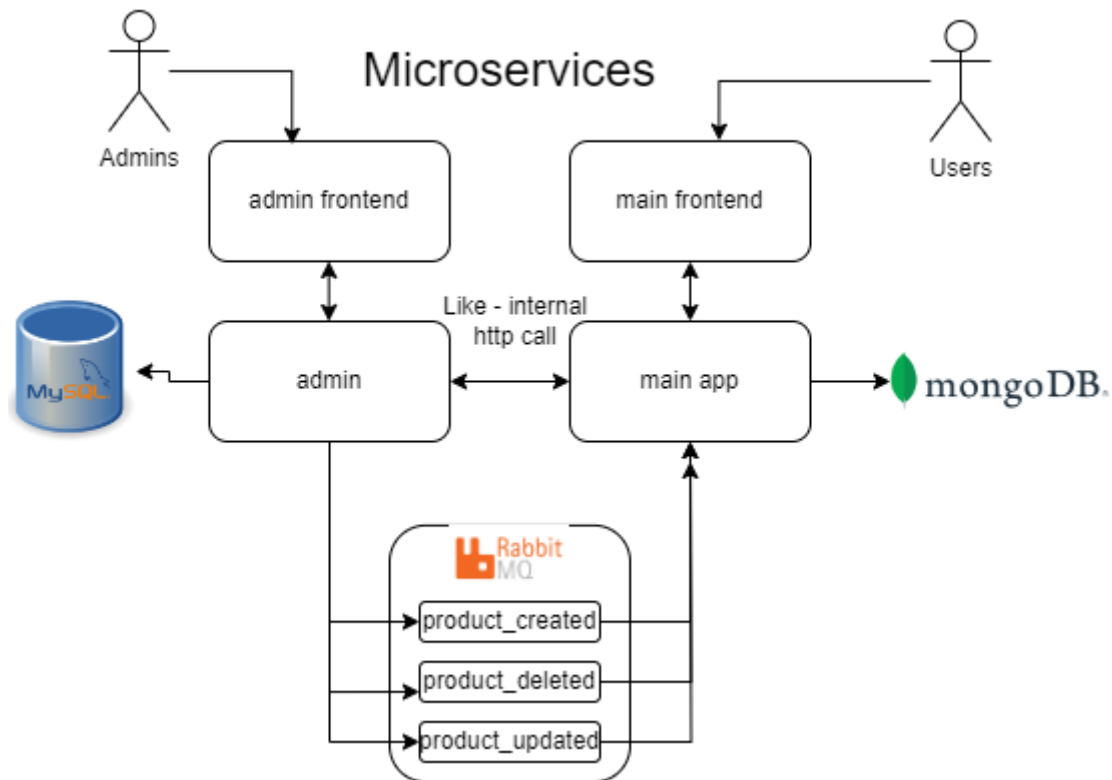
This is a simple demo of microservice architecture. It features a simple web shop that has products and it also has some admin functionality to create, update and delete a product by admin.

Monolithic architecture

If this application was a monolith, the architecture would look like this



Microservice architecture



Here, we moved from monolith into microservice architecture. We have 2 microservices both consisting of backend and frontend.

Message queues are used for synchronization of the two databases. Admin makes changes to MySQL and the main microservice gets notified to make the same changes on MongoDB.

The like synchronization is done via HTTP to show the difference between these 2 types of communication.

Testing the application

Run

```
docker compose up -d
```

Testing the asynchronous communication via message broker

If you stop main service, you can still work with the admin part and do the CRUD operations on products. The messages will be waiting in the message queues.

Once you start the main app again, it will consume all the messages one by one and thus the MongoDB database will be synchronized with the changes that were made by admin on MySQL database.

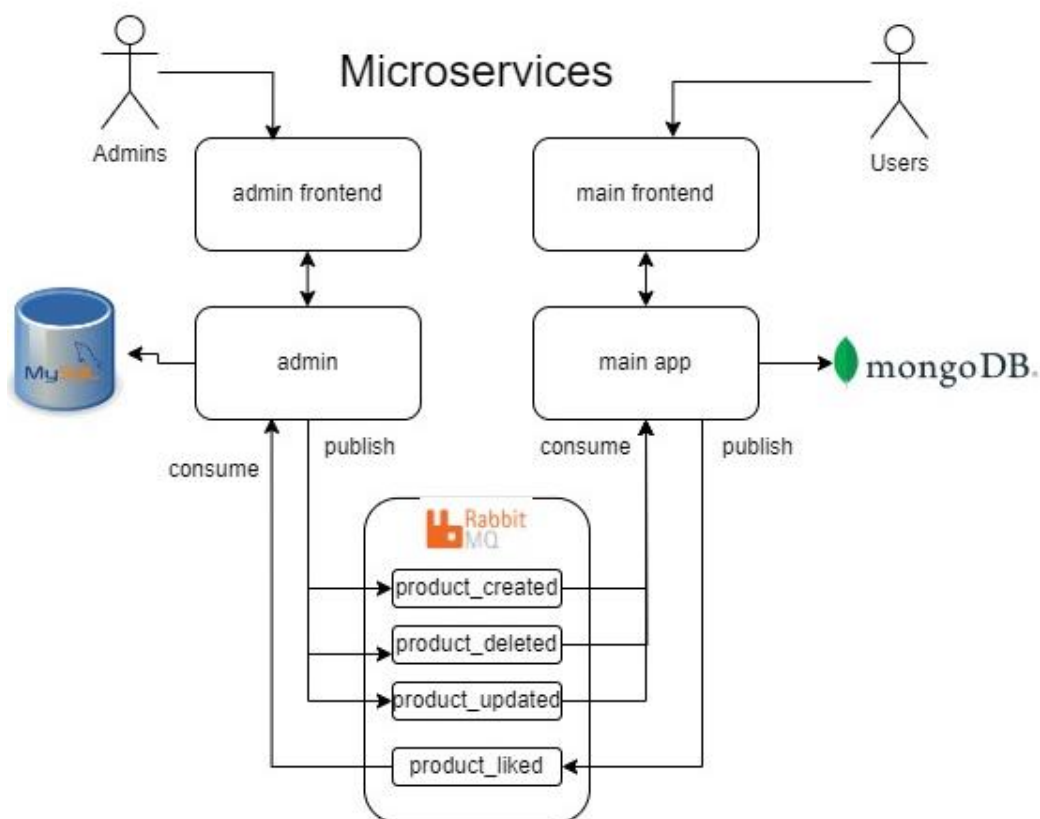
Testing the synchronous communication via http calls

A user can like a product. MongoDB database will be updated ($\text{liked} = \text{liked} + 1$). The synchronization with the admin database is done via internal http call. The main app will send http request to the admin.

If the admin service is down, the http request will fail and as a result the databases are not in sync. The like count in MongoDB is different from MySQL.

This is an example why http communication is not as robust as using message queues and it can lead to a loss of information, in this case it will lead to inconsistencies between the 2 databases.

Replacing HTTP call for Like with message queue



We added a `product_liked` queue. Now the main is the publisher and admin is the consumer. If the admin is down, the like messages will accumulate in the queue until the admin is back up. Then they will be consumed, and the MySQL database will become eventually consistent with the MongoDB database which was already updated when the “like” button was pressed.

This solution is more robust, because it prevents the inconsistency between the databases. Even if the admin is temporarily down, eventually, both databases will have the same data.

Scaling and load balancing

(more info is on github in the Readme file)

Run the application in local Kubernetes cluster:

```
kubectl apply -f .\kubernetes\
```

Port forward the main backend:

```
kubectl port-forward svc/main 8001:8001
```

Now scale the main backend:

```
kubectl scale deployment main --replicas=5
```

You should see 5 replicas in Docker desktop:

●	k8s_main_main-6686f954cc-cj9z	be4a3eac4fee	pesekt1/rabbitmq-microservices-main
●	k8s_main_main-6686f954cc-cx8v	f8b28ba1665b	pesekt1/rabbitmq-microservices-main
●	k8s_main_main-6686f954cc-smc	45f699420e70	pesekt1/rabbitmq-microservices-main
●	k8s_main_main-6686f954cc-xtdz	d12b2f556e48	pesekt1/rabbitmq-microservices-main
●	k8s_main_main-6686f954cc-zxlh	44b8b6eb860b	pesekt1/rabbitmq-microservices-main

Open the root api in the browser:

<http://localhost:8001/>

You should see the message containing the name of the pod:

Hello from main service on kubernetes pod: main-6686f954cc-smc98

You can try to refresh and see if you are redirected to different pods. Alternatively, delete the pod and refresh the page and you will be automatically redirected to another healthy pod.